

Esame OS Internals (Cabodi)

Data: 02/09/2024

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e IPT)

Q1) Rispondere YES/NO a ognuna delle domande seguenti:

1. In un sistema di memoria virtuale con un grado fisso di multiprogrammazione, il thrashing può verificarsi anche se la somma dei working set di tutti i processi è inferiore alla memoria fisica disponibile, ma l'algoritmo di sostituzione della pagina non è ottimizzato?
2. In un sistema con paginazione basata su inverted page table (IPT), è possibile che più pagine virtuali di processi diversi condividano una singola entry della tabella delle pagine da senza causare un conflitto nella traduzione degli indirizzi?
3. In un sistema di paginazione a richiesta che utilizza la sostituzione delle pagine locale (a un singolo processo), un processo con frequenza di page fault molto elevata può comunque causare una riduzione della memoria fisica disponibile per altri processi?

(Nota: Per ognuna delle risposte alle domande precedenti, occorre fornire una breve motivazione nella domanda successiva Q2).

Q3) Calcoli su IPT:

Si consideri un sistema a 64 bit con uno spazio di indirizzi virtuale di 2^{48} Byte, una dimensione di pagina di 4 KB e 16 GB di memoria fisica. Si supponga che il sistema utilizzi una inverted page table (IPT). Calcolare e spiegare quanto segue:

1. Il numero di bit necessari per il numero di frame fisico.
 2. Il numero totale di entry nella inverted page table.
 3. L'indirizzo virtuale assegnato a un processo è `0x00007FFFFFFFFF000`. Determinare l'indirizzo fisico se questo indirizzo virtuale è mappato al frame fisico 1024. Mostrare tutti i passaggi, incluso il calcolo del numero di pagina e dell'offset.
-

Domanda 2 (File System e Allocazione Linked)

Q4) Allocazione file:

Si consideri un file system che utilizza l'allocazione di tipo linked per l'allocazione dei file. Il disco è diviso in blocchi di 4 KB ciascuno. Ogni blocco contiene un puntatore al blocco successivo, che occupa 4 byte del blocco. Un file richiede 5 MB di spazio di archiviazione.

- A.** Calcolare il numero di blocchi del disco necessari per allocare l'intero file, incluso lo spazio necessario per i puntatori.

- A. Se i puntatori sono stati ridotti a 2 byte (presupponendo una capacità del disco inferiore), ricalcolare il numero di blocchi necessari. Questa modifica renderebbe l'archiviazione dei file più efficiente in termini di utilizzo dei blocchi?
- A. Confrontare il sovraccarico introdotto dai puntatori in entrambi i casi come percentuale dello spazio di archiviazione totale usato.
-

Domanda 3 (I/O, Interrupt, DMA e HDD)

Q5) Rispondere YES/NO a ognuna delle domande seguenti:

1. In un sistema con più dispositivi di I/O e una singola CPU, le operazioni di I/O possono essere eseguite in parallelo se la CPU è occupata nell'esecuzione di un processo?
2. In un sistema che utilizza I/O basato su interrupt, è possibile che un interrupt venga perso se il controller di interrupt è occupato nell'elaborazione di un altro interrupt e il dispositivo che ha attivato il secondo interrupt non supporta l'accodamento degli interrupt?
3. In un sistema con memoria virtuale basata su paginazione a richiesta e DMA, è possibile che si verifichi un page fault durante un trasferimento DMA, causando l'esito negativo del trasferimento se non vengono prese le dovute precauzioni?

(Nota: Per ognuna delle risposte alle domande precedenti, occorre fornire una breve motivazione nella domanda successiva Q6).

Q7) Calcoli su HDD:

Prendere in considerazione un'unità disco rigido (HDD) con le seguenti specifiche:

- Dimensione del settore: 512 byte;
 - Numero di tracce per faccia: 5.000;
 - Numero di settori per traccia: 300;
 - Numero di piatti a doppia faccia: 6;
 - Velocità di rotazione del piatto: 7.200 giri/min (giri al minuto).
1. Calcolare la velocità di trasferimento dati massima possibile in megabyte al secondo (MB/s), supponendo che sia possibile trasferire una traccia di dati per giro.
 2. Se il disco subisce un arresto anomalo della testina su un piatto, in che modo ciò influisce sulla capacità totale e sulla disponibilità dei dati supponendo che non siano in atto meccanismi RAID o di backup?
 3. Se il disco ha un tempo di ricerca medio di 4 ms e deve leggere un file da 1 GB suddiviso in 200 tracce non contigue, calcolare il tempo totale necessario per leggere il file. Includi il tempo per la ricerca, la latenza rotazionale e il trasferimento dei dati. Si supponga che la latenza rotazionale media sia di 4,165 ms e che sia possibile leggere una traccia per giro.

2 SECTION 2: OS161

Domanda 4 (dumbvm e Address Translation)

Contesto: Si consideri un sistema OS161, in esecuzione su un simulatore MIPS sys161 con 8 MB di memoria RAM.

Q8) Funzione `ram_stealmem`:

La memoria fisica in `dumbvm` viene allocata chiamando la funzione `ram_stealmem`. Per ciascuna delle seguenti frasi, rispondere vero/falso (true/false: non è necessaria alcuna spiegazione):

1. Sebbene la soluzione proposta per supportare la deallocazione della memoria (`freeppages`) sia basata su una bitmap, potrebbe essere utilizzata anche una lista concatenata.
2. La funzione `ram_stealmem` alloca in modo incrementale la memoria a partire dal primo indirizzo disponibile dopo aver fatto load del kernel.
3. Quando si utilizza una bitmap per tenere traccia della RAM allocata/libera, è necessario un secondo array che memorizzi le dimensioni delle partizioni allocate per l'allocazione sia della memoria dinamica del kernel che dei processi utente.
4. Lo stack di un processo utente non viene allocato (in `dumbvm`) chiamando `ram_stealmem`, perché la dimensione dello stack è una costante e lo stack si trova sempre agli stessi indirizzi logici.
5. La funzione `ram_stealmem` viene chiamata per l'allocazione della memoria sia del kernel (memoria dinamica) che dei processi user.
6. Lo schema di allocazione utilizzato da `ram_stealmem` non consentirà l'intercalare partizioni contigue del kernel e dei processi user (in altre parole, le partizioni del kernel e le partizioni user non saranno mai interlacciate).

Q9) Traduzione Indirizzi:

È noto che uno spazio di indirizzamento utente è caratterizzato da `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2`, `as->as_stackpbase`, aventi i seguenti valori: `0x200000`, `0x300000`, `0x3000`, `0x7000`, `3`, `4`, `0x400000`. Le pagine in OS161 hanno una dimensione di 4 KB. È anche noto che a uno stack user vengono allocate 18 pagine in `dumbvm`.

Dati i seguenti indirizzi logici (potrebbero essere sia user che kernel) convertirli nei relativi indirizzi fisici:

- `0x8110`
- `0x6500`
- `0x7FFFE010`
- `0x805000B0`

Domanda 5 (Syscalls e Debugging Lab4)

Q10) Funzione `sys_getpid`:

Considerare la funzione `sys_getpid` (prototipo sotto):

```
pid_t sys_getpid(void);
```

Implementato in Lab4. Per ciascuna delle seguenti frasi, rispondere vero/falso:

- La funzione è necessaria per implementare `sys_waitpid`, perché è necessario il pid del processo in attesa.
- La funzione è necessaria per implementare la chiamata di sistema `getpid`, e sarà chiamata dal kernel, nella funzione `syscall`.
- La funzione è necessaria per implementare la chiamata di sistema `getpid` e verrà chiamata direttamente da un processo utente.
- La funzione restituisce il pid del processo corrente.
- La funzione viene chiamata per ottenere il pid di un altro processo (quello di cui attendere la terminazione).
- La funzione viene chiamata da `sys_exit`, in modo da conoscere il pid del processo a cui inviare una segnalazione.

Q11) Debugging `proc_wait`:

Viene riportata una versione della funzione `proc_wait`, fornita all'interno della soluzione proposta nel laboratorio 4. La funzione contiene 3 errori. Trovarli e, per ognuno di essi, fornire una breve spiegazione (perché sono errori?).

```
int proc_wait(struct proc *proc) {
    int return_status;
    KASSERT (proc != NULL);
    KASSERT (proc != kproc);

    cv_wait(proc->p_cv, proc->p_lock);
    lock_acquire (proc->p_lock);
    return_status = proc->p_status;

    if (return_status < 0)
        return return_status;

    lock_release(proc->p_lock);
    proc_destroy (curproc);
    return return_status;
}
```

SOLUZIONI

Domanda 1 (Memoria Virtuale e IPT)

Risposte Q1:

1. **SÌ.** Il thrashing avviene quando il sistema spende più tempo a gestire page fault che a eseguire codice. Se l'algoritmo non è ottimizzato (es. LRU con un numero insufficiente di frame o algoritmi a sostituzione globale non controllata), esso può causare continui page fault su pagine attive del working set, portando al degrado anche se la RAM totale è sufficiente.
2. **SÌ.** In una IPT, ogni entry corrisponde a un frame fisico. La traduzione avviene tramite una tabella di hash che usa (pid, vp) come chiave. Poiché l'entry è legata al frame, non c'è conflitto tra pagine di processi diversi; l'entry identifica univocamente quale processo possiede quel frame in quel momento.
3. **NO.** Con una politica di sostituzione locale, un processo può rubare frame solo all'interno del proprio set di frame assegnati. Non può impattare la disponibilità di memoria fisica assegnata ad altri processi.

Q3) Calcoli su IPT:

1. **Bit per Frame Fisico (PFN):** Il numero di frame totali è dato dal rapporto tra la dimensione della RAM fisica e la dimensione di una singola pagina (frame):

$$\text{Numero di Frame} = \frac{\text{RAM}}{\text{Page Size}} = \frac{16 \text{ GB}}{4 \text{ KB}} = \frac{2^{34} \text{ Byte}}{2^{12} \text{ Byte}} = 2^{22} \text{ frame}$$

Per indirizzare univocamente 2^{22} frame, sono necessari esattamente **22 bit**.

2. **Entry nella IPT:** La Inverted Page Table (IPT) ha una struttura che scala in base alla memoria fisica, non allo spazio di indirizzamento virtuale. Deve esserci una entry per ogni frame fisico presente nel sistema.

$$\text{Numero di Entry} = \text{Numero di Frame} = 2^{22} = \mathbf{4.194.304} \text{ entry.}$$

3. **Calcolo Indirizzo Fisico:** Dato l'indirizzo virtuale 0x00007FFFFFFFFF000 e il mapping al frame fisico 1024 (0x400):

- **Estrazione Offset:** Poiché la pagina è di 4 KB (2^{12}), l'offset corrisponde ai 12 bit meno significativi dell'indirizzo virtuale: 0x000.
- **Identificazione PFN:** Il testo specifica che il VPN è mappato al frame 1024, ovvero 0x400 in esadecimale.
- **Composizione Indirizzo Fisico:** L'indirizzo fisico si ottiene concatenando il numero di frame fisico (PFN) e l'offset:

$$\text{Indirizzo Fisico} = (\text{PFN} \ll 12) \mid \text{Offset}$$

$$\text{Indirizzo Fisico} = (0x400 \ll 12) \mid 0x000 = 0x0000000000400000$$

Domanda 2 (File System e Allocazione Linked)

Risposte Q4:

- **Blocchi necessari:** File 5MB = $5 \cdot 1024 \cdot 1024 = 5.242.880$ byte. Blocco utile $4096 - 4 = 4092$ byte. Totale $\lceil 5242880/4092 \rceil = \mathbf{1282}$ blocchi.
- **Riduzione puntatori:** Blocco utile $4096 - 2 = 4094$ byte. Totale $\lceil 5242880/4094 \rceil = \mathbf{1281}$ blocchi. L'efficienza aumenta.
- **Confronto sovraccarico:** Caso 4 byte $\approx 0.097\%$; Caso 2 byte $\approx 0.048\%$.

Domanda 3 (I/O, Interrupt, DMA e HDD)

Risposte dettagliate Q5:

1. **I/O in Parallelo:** *SÌ*. I moderni dispositivi di I/O sono dotati di propri controller (come il controller DMA) che gestiscono il trasferimento dei dati in modo indipendente. Una volta che la CPU impartisce il comando di inizio I/O, il controller lavora in parallelo accedendo alla memoria o al dispositivo, lasciando la CPU libera di eseguire altri processi.
2. **Interrupt Persi:** *SÌ*. Se la CPU (o il controller degli interrupt) sta già servendo un interrupt critico (avendo temporaneamente mascherato/disabilitato gli altri interrupt) e arriva un secondo segnale da un dispositivo privo di buffer/coda per gli interrupt, questo nuovo segnale può essere sovrascritto o ignorato, venendo di fatto perso.
3. **Page Fault in DMA:** *SÌ*. Il controller DMA lavora direttamente con gli indirizzi fisici scavalcando la CPU. Se il DMA tenta di trasferire dati in una pagina di memoria che è stata temporaneamente spostata su disco (swap-out), causerà un errore grave poiché il DMA non è in grado di gestire un page fault. Per questo motivo, le pagine coinvolte nel DMA devono essere "bloccate" (pinned/locked) in RAM dal Sistema Operativo prima di iniziare il trasferimento.

Calcoli HDD (Q7):

1. Velocità di Trasferimento Dati Massima (Transfer Rate):

- **Dati per traccia:** Ogni traccia ha 300 settori da 512 byte.

$$\text{Capacità Traccia} = 300 \times 512 \text{ byte} = 153.600 \text{ Byte} = 150 \text{ KB}$$

- **Giri al secondo (Hz):** Il disco ruota a 7.200 RPM.

$$\text{Giri per secondo} = \frac{7.200}{60} = 120 \text{ giri/secondo (Hz)}$$

- **Transfer Rate:** Potendo leggere una traccia intera per ogni giro, la velocità è:

$$\text{Transfer Rate} = 153.600 \text{ Byte/giro} \times 120 \text{ giri/s} = 18.432.000 \text{ Byte/s}$$

In Megabyte (assumendo $1 \text{ MB} = 10^6 \text{ Byte}$ per convenzione dei produttori storage): **18,432 MB/s**. (Oppure $\approx 17,58 \text{ MiB/s}$ se calcolato diviso 1024^2).

2. Effetto dell'Head Crash su un piatto:

- **Capacità Totale:** Il disco ha 6 piatti a doppia faccia, per un totale di 12 facce (12 testine). Un head crash distrugge la superficie di una singola faccia. La perdita di capacità è pari a $\frac{1}{12}$ del totale (circa l'**8,33%**).

- **Disponibilità dei Dati:** Senza sistemi RAID o backup, tutti i dati residenti fisicamente su quella specifica faccia sono irrimediabilmente perduti. Poiché i file moderni sono spesso frammentati sui vari piatti per parallelizzare la lettura (cylinder concept), un danno a una singola faccia corrompe la stragrande maggioranza dei file di grandi dimensioni sul disco.

3. Tempo di Lettura file da 1 GB:

- *Nota sul testo:* 1 GB di dati equivale a circa 10^9 Byte. Poiché una traccia contiene solo 153.600 Byte, 1 GB occuperebbe circa 6.510 tracce. L'espressione "suddiviso in 200 tracce non contigue" indica presumibilmente che il file è diviso in **200 frammenti** sparsi per il disco. Calcoliamo il tempo totale come la somma del tempo di trasferimento netto più l'overhead di posizionamento per i 200 salti.

- **Tempo di Trasferimento (Transfer Time):**

$$\text{Transfer Time} = \frac{1 \text{ GB}}{18,432 \text{ MB/s}} = \frac{1000 \text{ MB}}{18,432 \text{ MB/s}} \approx \mathbf{54,25} \text{ secondi}$$

- **Tempo di Ricerca Totale (Seek Time):**

$$\text{Total Seek Time} = 200 \times 4 \text{ ms} = 800 \text{ ms} = \mathbf{0,8} \text{ secondi}$$

- **Latenza Rotazionale Totale (Rotational Latency):**

$$\text{Total Rotational Latency} = 200 \times 4,165 \text{ ms} = 833 \text{ ms} = \mathbf{0,833} \text{ secondi}$$

- **Tempo Totale:**

$$\text{Total Time} = 54,25 + 0,8 + 0,833 = \mathbf{55,883} \text{ secondi}$$

Domanda 4 (dumbvm e Address Translation)

Risposte Q8/Q9:

- **Q6:** T, T, T, F, T, F.
- **Q7 (Indirizzi): Q7) Traduzione Indirizzi:**

Regola generale di traduzione: Indirizzo Fisico = Base Fisica (PStart)+(Indirizzo Virtuale−Base Virtuale (VStart)).

– Processo 1 (Codice e Dati):

- * 0x6500 (Segmento Codice / Pbase1): L'indirizzo rientra nel range virtuale del codice (da 0x3000 a 0x6FFF).

$$\text{Offset} = 0x6500 - 0x3000 = 0x3500.$$

$$\text{Indirizzo Fisico} = 0x200000 + 0x3500 = 0x203500.$$

- * 0x8110 (Segmento Dati / Pbase2): L'indirizzo rientra nel range virtuale dei dati (da 0x7000 a 0xBFFF).

$$\text{Offset} = 0x8110 - 0x7000 = 0x1110.$$

$$\text{Indirizzo Fisico} = 0x300000 + 0x1110 = 0x301110.$$

– Processo 2 (Stack):

* 0x7FFFE010 (Segmento Stack):

A differenza dei segmenti di codice e dati che crescono verso l'alto (da un indirizzo basso a uno alto), lo stack cresce verso il basso a partire da un tetto massimo fisso. In OS161, questo limite invalicabile è definito dalla costante `USERSTACK = 0x80000000`.

Passo 1: Calcolo della dimensione dello stack. Il testo specifica che sono allocate 18 pagine. Essendo ogni pagina di 4 KB (4096 byte = `0x1000`), la dimensione totale è $18 \times 0x1000 = 0x12000$ byte.

Passo 2: Calcolo della Base Virtuale minima (VStart). Sottraendo la dimensione complessiva dal tetto massimo, si ottiene l'indirizzo virtuale più basso valido per lo stack:

$$0x80000000 - 0x12000 = 0x7FFEE000.$$

Passo 3: Calcolo dell'Offset. Si calcola la distanza dell'indirizzo richiesto dalla base virtuale appena trovata:

$$0x7FFFE010 - 0x7FFEE000 = 0x10010.$$

Passo 4: Mappatura Fisica. Si aggiunge l'offset alla base fisica fornita dal testo (`as->as_stackbase`):

$$0x400000 + 0x10010 = 0x410010.$$

– **Indirizzi Kernel:**

* 0x805000B0 (Indirizzo Kernel in `kseg0`):

Gli indirizzi kernel residenti nel segmento `kseg0` (che va da `0x80000000` a `0x9FFFFFFF`) godono di una traduzione diretta via hardware nel processore MIPS. Si ottiene l'indirizzo fisico semplicemente sottraendo l'offset di base del segmento (`0x80000000`).

$$\text{Indirizzo Fisico} = 0x805000B0 - 0x80000000 = 0x005000B0.$$

Domanda 5 (Syscalls e Debugging Lab4)

Risposte Q8 (Funzione `sys_getpid`):

- **FALSO.** `sys_waitpid` riceve il PID del processo da attendere come parametro in ingresso (tramite `trapframe`). Non ha bisogno di `sys_getpid()` per conoscere il PID del chiamante o dell'atteso.
- **VERO.** Le applicazioni in User Space chiamano la funzione di libreria `getpid()`. Questo genera una trap che porta il flusso di esecuzione nel kernel alla funzione dispatcher `syscall()`, la quale a sua volta decodifica la richiesta e chiama la routine kernel `sys_getpid()`.
- **FALSO.** Un processo utente (User Mode) non può MAI chiamare direttamente funzioni residenti in memoria Kernel. L'unico modo per attraversare la barriera dei privilegi è tramite un'eccezione hardware (`trap/syscall`).
- **VERO.** La funzione, non prendendo argomenti (prototipo `void`), accede semplicemente alla struttura del processo attualmente in esecuzione (es. tramite `curproc`) per restituirne l'identificativo.
- **FALSO.** La chiamata serve esclusivamente a ottenere il proprio PID. Non c'è modo di passargli un puntatore o un handle per ispezionare processi terzi.
- **FALSO.** Il kernel all'interno di `sys_exit` ha già visibilità completa e diretta sulla struttura `curproc`. Non ha bisogno di chiamare `sys_getpid()` per recuperare informazioni del processo corrente.

Q9) Debugging `proc_wait`:

La funzione presenta i seguenti tre errori logici critici:

1. **Ordine errato tra Lock e Condition Variable:**

`cv_wait` è chiamato prima di `lock_acquire`. Per definizione, un thread deve già possedere il lock associato prima di potersi mettere in attesa su una Condition Variable. Eseguire `cv_wait` senza possedere il lock causa un errore a runtime (panic).

2. **Mancato rilascio del Lock (Deadlock):**

All'interno dell'istruzione `if (return_status < 0)`, la funzione ritorna immediatamente il valore di errore. Tuttavia, la funzione omette di chiamare `lock_release(proc->p_lock)` prima del `return`. Questo lascia il lock acquisito in modo permanente, causando un deadlock per chiunque tenti di accedervi in futuro.

3. **Distruzione del processo errato (curproc):**

L'istruzione `proc_destroy(curproc)` ordina la distruzione del processo correntemente in esecuzione (il processo Padre che ha invocato la `wait`). La semantica corretta di una `wait` impone invece di deallocare e distruggere le strutture dati del processo Figlio terminato. Il comando corretto avrebbe dovuto essere `proc_destroy(proc)`.